CS Concepts Cheat Sheet — Last-Minute Revision

Covers: OOP · DSA · OS · DBMS · Computer Networks · AI & Data Science

---

1. Object-Oriented Programming (OOP)

The 4 Pillars — Mnemonic: A PIE

Pillar	Definition	Example
Abstraction	Hide complexity, expose only essentials	Abstract class `Shape` with `area()` — caller doesn't care *how*
Polymorphism	One interface, many implementations	`draw()` behaves differently for `Circle` vs `Square`
Inheritance	Child class acquires properties of parent (IS-A)	`Dog extends Animal`
Encapsulation	Bundle data + methods; restrict direct access	Private fields + public getters/setters

Key Definitions

- **Class**: Blueprint/template. **Object**: Instance of a class (has state + behavior + identity).
- **Constructor**: Special method invoked at object creation; no return type; can be overloaded.
- **`static`**: Belongs to the class, not instances; shared across all objects; no `this`.
- **`final` (Java) / `const` / `sealed` (C#)**: Prevents modification/inheritance/overriding.
- **Interface**: Pure contract (all abstract by default; Java 8+ allows `default` methods). A class can implement many.
- **Abstract class**: Can mix abstract + concrete methods, can have state/constructors. Single inheritance only (Java/C#).

Abstract Class vs Interface

	Abstract Class	Interface
Methods	Abstract + concrete	Abstract (+ default/static in Java 8+)
Fields	Instance variables allowed	Only `public static final` constants (Java)
Inheritance	Extend ONE	Implement MANY
Constructor	Yes	No
Use when	Shared code + IS-A hierarchy	Capability contract (CAN-DO: `Comparable`, `Serializable`)

Polymorphism: Compile-time vs Runtime

	Overloading (compile-time / static)	Overriding (runtime / dynamic)
Signature	Same name, **different parameters**	Same name, **same signature**
Where	Same class	Parent → child class
Binding	Early (compiler resolves)	Late (virtual dispatch via vtable)
Return type	Can differ	Same or covariant
`static`/`private`/`final` methods	Can be overloaded	**Cannot** be overridden

Pitfall: In Java, overriding resolution uses the *object's runtime type*; overloading uses the *reference's declared type*.

Relationships

- **IS-A** → Inheritance (`Car IS-A Vehicle`)
- **HAS-A** → Composition/Aggregation
  - **Aggregation**: weak HAS-A; part survives whole (Department–Teacher)
  - **Composition**: strong HAS-A; part dies with whole (House–Room)
- **Rule of thumb**: *Favor composition over inheritance* (looser coupling, avoids fragile base class problem).

SOLID Principles

Letter	Principle	One-liner
S	Single Responsibility	A class = one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subtype must be usable wherever parent is (classic violation: Square extends Rectangle)
I	Interface Segregation	Many small interfaces > one fat interface
D	Dependency Inversion	Depend on abstractions, not concretions (basis of Dependency Injection)

Common Design Patterns (interview favorites)

Pattern	Category	Purpose
Singleton	Creational	One instance globally (DB connection pool, logger). Pitfall: thread safety → use double-checked locking or enum (Java)
Factory Method	Creational	Delegate object creation; hide `new`
Builder	Creational	Step-by-step construction of complex objects
Adapter	Structural	Convert one interface to another (legacy integration)
Decorator	Structural	Add behavior dynamically without subclassing (Java I/O streams)
Observer	Behavioral	Publish–subscribe; one-to-many notification (event listeners)
Strategy	Behavioral	Swap algorithms at runtime (payment methods, sorting comparators)

Other FAQ Topics

- **Shallow vs deep copy**: shallow copies references (shared nested objects); deep clones the whole graph.
- **Constructor vs method**: constructor has no return type, name = class name, auto-invoked.
- **`this` vs `super`**: current object vs immediate parent.
- **Method hiding**: static methods with same signature in child "hide" (not override) parent's.
- **Diamond problem**: multiple inheritance ambiguity — Java bans class multiple inheritance; C++ uses `virtual` inheritance; Java 8 default-method conflicts must be overridden explicitly.
- **Coupling vs Cohesion**: aim for **low coupling** (few dependencies between classes) and **high cohesion** (class does one focused job).

---

2. Data Structures & Algorithms (DSA)

Asymptotic Notation

- **Big-O (O)**: upper bound (worst case) — the one interviewers mean by default.
- **Omega (Ω)**: lower bound (best case). **Theta (Θ)**: tight bound.
- Growth order: `O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(n³) < O(2ⁿ) < O(n!)`
- **Amortized analysis**: average per-operation over a sequence (e.g., dynamic array append = amortized O(1) despite occasional O(n) resize).

Data Structure Complexities

Structure	Access	Search	Insert	Delete	Notes
Array	O(1)	O(n)	O(n)	O(n)	Contiguous memory; cache-friendly
Dynamic Array	O(1)	O(n)	O(1)* amortized (end)	O(n)	Doubles capacity on resize
Singly Linked List	O(n)	O(n)	O(1)†	O(1)†	† at known node/head
Doubly Linked List	O(n)	O(n)	O(1)†	O(1)†	Extra prev pointer
Stack (LIFO)	O(n)	O(n)	O(1) push	O(1) pop	Undo, call stack, DFS, expression eval
Queue (FIFO)	O(n)	O(n)	O(1) enqueue	O(1) dequeue	BFS, scheduling, buffering
Hash Table	—	O(1) avg / O(n) worst	O(1) avg	O(1) avg	Worst = all collisions
BST (unbalanced)	O(h)	O(h)	O(h)	O(h)	h = height; worst O(n) if skewed
Balanced BST (AVL/Red-Black)	O(log n)	O(log n)	O(log n)	O(log n)	AVL stricter balance → faster lookup, slower insert
Binary Heap	O(1) peek	O(n)	O(log n)	O(log n) extract	Priority queues; heapify array in O(n)
Trie	—	O(L)	O(L)	O(L)	L = key length; prefix search, autocomplete
Graph (adj. list)	—	O(V+E) traverse	O(1) add edge	—	Sparse graphs; adjacency matrix = $O(V^2)$ space, O(1) edge check

Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Best case = already sorted with early-exit flag
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Minimum swaps (n−1)
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Great for small/nearly-sorted data

Algorithm	Best	Average	Worst	Space	Stable?	Notes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Guaranteed n log n; external sorting; linked lists
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No	Worst = bad pivot (sorted input + first-element pivot); fastest in practice
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No	In-place guaranteed n log n
Counting	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes	Non-comparison; small integer range k
Radix	$O(d \cdot (n+k))$	—	—	O(n+k)	Yes	Digit-by-digit using counting sort

- **Comparison-sort lower bound**: $\Omega(n \log n)$ — proven via decision tree.
- **Stable** = equal elements keep relative order (matters for multi-key sorts).

Searching

- **Linear search**: O(n), unsorted OK.
- **Binary search**: O(log n), requires sorted data. Pitfall: `mid = low + (high-low)/2` avoids overflow; off-by-one boundary errors are the classic bug.

Graph Algorithms

Algorithm	Purpose	Complexity	Notes
BFS	Level-order traversal; shortest path (unweighted)	O(V+E)	Queue
DFS	Traversal; cycle detection; topological sort	O(V+E)	Stack/recursion
Dijkstra	Shortest path, **non-negative** weights	O((V+E) log V) with min-heap	Fails on negative edges
Bellman–Ford	Shortest path, negative edges allowed	$O(V \cdot E)$	Detects negative cycles
Floyd–Warshall	All-pairs shortest paths	$O(V^3)$	DP over intermediate vertices
Kruskal (MST)	Minimum spanning tree	O(E log E)	Sort edges + Union-Find
Prim (MST)	Minimum spanning tree	O(E log V)	Grow tree from a vertex; better for dense graphs
Topological Sort	Order DAG by dependencies	O(V+E)	Kahn's (in-degree/BFS) or DFS finish times; only on DAGs

Tree Traversals — Mnemonic: position of Root (Pre = Root first, In = Root in middle, Post = Root last)

- **Inorder (L, Root, R)** → sorted output for BST
- **Preorder (Root, L, R)** → copy/serialize tree
- **Postorder (L, R, Root)** → delete tree, evaluate expression trees
- **Level order** → BFS with queue

Algorithmic Paradigms

Paradigm	Idea	Classic Problems
Divide & Conquer	Split → solve → combine	Merge sort, quicksort, binary search
Greedy	Locally optimal choice each step	Activity selection, Huffman coding, Dijkstra, Kruskal
Dynamic Programming	Overlapping subproblems + optimal substructure; memoize	Fibonacci, knapsack (0/1), LCS, LIS, coin change, edit distance
Backtracking	Try, fail, undo	N-Queens, Sudoku, subsets/permutations
Two Pointers / Sliding Window	Linear scans with moving bounds	Pair sum in sorted array, longest substring without repeats

- **Greedy vs DP**: greedy commits and never revisits (needs greedy-choice property); DP explores all subproblems. Fractional knapsack → greedy; 0/1 knapsack → DP.
- **Memoization (top-down)** vs **Tabulation (bottom-up)**: same complexity; tabulation avoids recursion overhead/stack depth.

Recurrences & Master Theorem

For `T(n) = a·T(n/b) + f(n)`, compare `f(n)` with `n^(log_b a)`: - f smaller → $T(n) = \Theta(n^{\log_b a})$ — e.g., binary search: a=1, b=2, f=O(1) → O(log n) - f equal → $\Theta(n^{\log_b a} \cdot \log n)$ — e.g., merge sort: a=2, b=2, f=n → O(n log n) - f larger (regularity holds) → $\Theta(f(n))$

Common Pitfalls & FAQ

- Hash map is O(1) *average*; adversarial inputs / bad hash → O(n). Java 8 HashMap converts long collision chains to red-black trees (O(log n)).
- Recursion depth → stack overflow; convert to iteration or increase stack for deep trees.
- BST validity must check *entire subtree ranges*, not just immediate children (classic interview trap).
- Detect cycle in linked list → Floyd's tortoise & hare, O(1) space.
- Union-Find with path compression + union by rank → near O(1) amortized (inverse Ackermann).

---

3. Operating Systems (OS)

Core Definitions

- **OS**: Resource manager + interface between hardware and user programs.
- **Kernel**: Core of the OS (process, memory, device, file management). **Monolithic** (Linux — fast, all in kernel space) vs **Microkernel** (Minix, QNX — minimal kernel, services in user space, more reliable/slower) vs **Hybrid** (Windows, macOS).
- **System call**: User program's request to the kernel (mode switch user → kernel). Examples: `fork`, `exec`, `read`, `write`, `open`.
- **Process**: Program in execution (own address space). **Thread**: Lightweight unit within a process (shares code/data/heap; own stack + registers).

Process vs Thread

	Process	Thread
Memory	Separate address space	Shared within process
Creation/switch cost	Heavy	Light
Communication	IPC (pipes, shared memory, message queues, sockets)	Shared variables (need synchronization)
Fault isolation	One crash doesn't kill others	One bad thread can crash the process

Process States

`New → Ready → Running → (Waiting/Blocked on I/O → Ready) → Terminated` - **Context switch**: Save/restore PCB (Process Control Block); pure overhead. - **Zombie**: terminated child whose parent hasn't called `wait()`. **Orphan**: parent died first (adopted by init/systemd).

CPU Scheduling

Algorithm	Type	Notes
FCFS	Non-preemptive	Simple; convoy effect (long job blocks short ones)
SJF / SRTF	Non-preemptive / Preemptive	Optimal average waiting time; needs burst prediction; starvation of long jobs
Priority	Either	Starvation risk → fix with **aging**
Round Robin	Preemptive	Time quantum q; too small → switch overhead, too large → FCFS. Fair, good response time
Multilevel (Feedback) Queue	Preemptive	Queues by type/priority; feedback allows movement between queues

Formulas: Turnaround = Completion − Arrival · Waiting = Turnaround − Burst · Response = First-CPU − Arrival

Synchronization

- **Race condition**: Outcome depends on interleaving of concurrent access to shared data.
- **Critical section requirements**: Mutual exclusion, Progress, Bounded waiting.
- **Mutex**: binary lock, ownership (locker must unlock). **Semaphore**: counter; binary or counting (resource pool); `wait(P)`/`signal(V)`. **Monitor**: high-level construct (synchronized methods + condition variables).
- **Spinlock**: busy-waits — fine for very short waits on multicore; wasteful otherwise.
- Classic problems: **Producer–Consumer** (bounded buffer), **Readers–Writers**, **Dining Philosophers**.

Deadlock — 4 Coffman Conditions (all must hold) — Mnemonic: M-H-N-C

1. **M**utual exclusion 2. **H**old and wait 3. **N**o preemption 4. **C**ircular wait

- **Handling**: Prevention (break a condition, e.g., ordered resource acquisition kills circular wait), Avoidance (**Banker's algorithm** — grant only if state stays safe), Detection + recovery (wait-for graph), Ostrich (ignore — most OSes).
- **Deadlock vs Starvation**: deadlock = nobody progresses (circular block); starvation = someone never progresses (unfair scheduling).

Memory Management

- **Paging**: fixed-size frames/pages; eliminates external fragmentation; suffers **internal** fragmentation. Page table maps virtual → physical; **TLB** caches translations.
- **Segmentation**: variable-size logical units; external fragmentation.
- **Virtual memory**: run programs larger than RAM via demand paging.
- **Page fault**: referenced page not in RAM → load from disk.
- **Thrashing**: excessive paging, CPU mostly waits; fix via working-set model / fewer processes.
- **EAT** = (1−p) · mem_access + p · page_fault_time (p = fault rate)

Page Replacement

Algorithm	Idea	Notes
FIFO	Evict oldest	**Belady's anomaly**: more frames can → more faults
Optimal (OPT)	Evict page used farthest in future	Theoretical benchmark only
LRU	Evict least recently used	Good approximation of OPT; costly to implement exactly → clock/second-chance approximations

Allocation strategies (contiguous)

First fit (fast) · Best fit (smallest hole — leaves tiny slivers) · Worst fit (largest hole).

File Systems & Disk

- Allocation: contiguous (fast, fragmentation), linked (no random access), **indexed** (inode — UNIX).
- Disk scheduling: FCFS, **SSTF** (starvation risk), **SCAN/elevator**, C-SCAN (uniform wait), LOOK/C-LOOK.
- RAID: 0 stripe (speed, no redundancy) · 1 mirror · 5 stripe+parity (1 disk failure) · 6 dual parity · 10 mirror+stripe.

FAQ / Pitfalls

- `fork()` returns 0 in child, child PID in parent; classic "how many processes does n forks create?" $\rightarrow 2^n$.
- **Preemptive vs non-preemptive**: can the OS forcibly take the CPU away?
- **Paging vs Segmentation**: physical fixed blocks vs logical variable units.
- **User-level vs kernel-level threads**: user threads fast but one blocking syscall blocks all (many-to-one); kernel threads schedulable independently.
- **Belady's anomaly** applies to FIFO, not LRU/OPT (stack algorithms).

---

4. Database Management Systems (DBMS)

Core Concepts

- **DBMS advantages over file systems**: reduced redundancy, consistency, concurrent access, security, recovery, data independence.
- **Three-schema architecture**: External (views) $\rightarrow$ Conceptual (logical) $\rightarrow$ Internal (physical). **Data independence**: change lower level without affecting upper.
- **Keys**:
  - **Super key**: any attribute set that uniquely identifies rows
  - **Candidate key**: minimal super key
  - **Primary key**: chosen candidate key (NOT NULL + UNIQUE)
  - **Alternate key**: candidate keys not chosen
  - **Foreign key**: references PK of another table (referential integrity)
  - **Composite key**: multi-column key
- **Oracle note**: PK automatically creates a unique index; FKs do NOT auto-create indexes — unindexed FKs cause locking issues and slow deletes on parent tables.

Normalization — "The key, the whole key, and nothing but the key"

Normal Form	Rule	Removes
1NF	Atomic values, no repeating groups	Multivalued cells
2NF	1NF + no **partial dependency** (non-key attr depends on part of composite key)	Partial dependencies
3NF	2NF + no **transitive dependency** (non-key $\rightarrow$ non-key)	Transitive dependencies
BCNF	For every FD X$\rightarrow$Y, X is a super key	Anomalies 3NF can miss (overlapping candidate keys)

- **Anomalies solved**: insertion, update, deletion anomalies.
- **Denormalization**: deliberate redundancy for read performance (reporting/warehouse). Trade-off: faster reads vs update anomalies + storage.

SQL Quick Reference

- **Order of execution**: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT` (write order ≠ run order — why you can't use SELECT aliases in WHERE).
- **WHERE vs HAVING**: WHERE filters rows before grouping; HAVING filters groups after aggregation.
- **DELETE vs TRUNCATE vs DROP**: DELETE = DML, row-by-row, WHERE allowed, rollback-able, fires triggers; TRUNCATE = DDL, deallocates all rows, fast, resets identity, (Oracle: cannot rollback); DROP removes the object entirely.
- **`UNION` vs `UNION ALL`**: UNION deduplicates (sort cost); UNION ALL keeps duplicates (faster).

- **Correlated subquery**: inner query references outer row → runs per row (often rewrite as JOIN for performance).
- **Window functions**: `ROW_NUMBER()` (unique), `RANK()` (gaps), `DENSE_RANK()` (no gaps) `OVER (PARTITION BY ... ORDER BY ...)` — classic "2nd highest salary" tool.
- 2nd highest salary (portable): `SELECT MAX(sal) FROM emp WHERE sal < (SELECT MAX(sal) FROM emp);`

Joins

Join	Returns
INNER	Only matching rows
LEFT OUTER	All left + matched right (NULLs for no match)
RIGHT OUTER	All right + matched left
FULL OUTER	Everything, matched where possible
CROSS	Cartesian product (m×n)
SELF	Table joined to itself (employee–manager)

Physical join methods (Oracle optimizer): **Nested Loops** (small driving set + indexed inner), **Hash Join** (large unsorted sets, equi-joins), **Sort-Merge** (both sorted / range joins).

Transactions — ACID

- **A**tomicity: all-or-nothing (undo/rollback)
- **C**onsistency: valid state → valid state (constraints hold)
- **I**solation: concurrent txns don't interfere
- **D**urability: committed = permanent (redo logs / WAL)

Isolation Levels vs Anomalies

Level	Dirty Read	Non-repeatable Read	Phantom
Read Uncommitted	× possible	×	×
Read Committed	✓ prevented	×	×
Repeatable Read	✓	✓	×
Serializable	✓	✓	✓

- **Oracle note**: default = Read Committed; Oracle never allows dirty reads (MVCC/undo gives statement-level read consistency); no explicit Repeatable Read — it offers Serializable and read-only transactions instead.
- **Concurrency control**: Two-Phase Locking (2PL: growing + shrinking phases → serializability, but deadlocks possible), timestamp ordering, **MVCC** (readers don't block writers — Oracle/PostgreSQL).
- **Recovery**: Write-Ahead Logging; checkpoints; undo (before-image) + redo (after-image).

Indexing

- **B+ Tree** (default nearly everywhere incl. Oracle): sorted, O(log n) lookups, great for range scans; leaves linked.
- **Hash index**: O(1) equality only, no ranges.
- **Bitmap index** (Oracle): low-cardinality columns in read-heavy warehouses; terrible for OLTP (locking).
- **Clustered** (data stored in index order — one per table) vs **Non-clustered** (separate structure pointing to rows). Oracle equivalent of clustered ≈ Index-Organized Table (IOT).
- **When indexes hurt**: heavy-write tables, low-selectivity columns, functions on indexed columns in WHERE (kills index use unless function-based index).
- **Composite index**: leftmost-prefix rule — index on (a,b,c) serves a / a,b / a,b,c but not b alone.

ER Model & Misc

- Entity, attribute (simple/composite/derived/multivalued), relationship cardinality (1:1, 1:N, M:N — M:N needs a junction table).
- **Weak entity**: no key of its own; identified via owner + partial key (double rectangle/diamond).
- **Stored procedure vs function** (Oracle/PLSQL): procedure may return many/no values via OUT params, called as statement; function returns one value, usable in SQL.
- **Trigger**: auto-fired on DML/DDL events. Pitfall: hidden logic, mutating-table errors (Oracle).

- **View**: virtual table (saved query). **Materialized view**: physically stored snapshot, refreshed — reporting workhorse in Oracle.
- **OLTP vs OLAP**: many short transactions, normalized vs analytical scans, star/snowflake schemas, denormalized.
- **CAP theorem** (distributed): can only guarantee 2 of Consistency, Availability, Partition tolerance. SQL ≈ CA/CP mindset; NoSQL often AP with eventual consistency.
- **SQL vs NoSQL**: fixed schema + joins + ACID vs flexible schema + horizontal scaling + BASE (Basically Available, Soft state, Eventual consistency).

---

5. Computer Networks (CN)

OSI 7 Layers — Mnemonic: "Please Do Not Throw Sausage Pizza Away" (bottom-up)

#	Layer	Unit (PDU)	Job	Devices/Protocols
7	Application	Data	User-facing services	HTTP, FTP, SMTP, DNS
6	Presentation	Data	Encryption, compression, formats	TLS/SSL, JPEG
5	Session	Data	Establish/manage sessions	NetBIOS, RPC
4	Transport	Segment	End-to-end delivery, ports	TCP, UDP
3	Network	Packet	Logical addressing, routing	IP, ICMP, routers
2	Data Link	Frame	MAC addressing, error detection	Ethernet, switches, ARP
1	Physical	Bits	Signals over medium	Cables, hubs, repeaters

TCP/IP model (4 layers): Application (7–5) → Transport → Internet → Network Access (2–1).

TCP vs UDP

	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	ACKs, retransmission, ordering, checksums	Best-effort, no ordering
Speed/overhead	Slower, 20-byte+ header	Faster, 8-byte header
Flow/congestion control	Yes (sliding window; slow start, AIMD)	No
Use cases	Web, email, file transfer, DB connections	DNS, VoIP, video streaming, gaming, DHCP

- **3-way handshake**: SYN → SYN-ACK → ACK. **Teardown**: FIN/ACK ×2 (4-way); TIME_WAIT on closer's side.
- **Congestion control**: slow start (exponential) → congestion avoidance (linear, AIMD) → on loss: fast retransmit/recovery (3 dup ACKs) or timeout (back to slow start).

Addressing

- **IPv4**: 32-bit dotted decimal; ~4.3B addresses. **IPv6**: 128-bit hex, no NAT needed, built-in IPsec support.
- **Private ranges** (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`.
- **Classes**: A (1–126), B (128–191), C (192–223); 127.x = loopback. Modern world uses **CIDR** (`/24` = 255.255.255.0).
- **Subnetting**: usable hosts = 2^(host bits) − 2 (network + broadcast). `/26` → 64 addresses, 62 hosts.
- **NAT**: many private IPs share public IP(s); breaks end-to-end but conserves IPv4.
- **MAC**: 48-bit, layer-2, burned into NIC; **ARP** maps IP → MAC (broadcast "who has X?").
- **DHCP**: auto-assign IP — DORA: **D**iscover, **O**ffer, **R**equest, **A**ck.

DNS

Hierarchical name → IP resolution: browser cache → OS cache → **recursive resolver** → root → TLD (.com) → authoritative server. Records: **A** (IPv4), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail), **NS**, **TXT**. Uses UDP 53 (TCP for zone transfers/large responses).

"What happens when you type a URL?" (top interview question)

1. DNS resolution → IP
2. TCP 3-way handshake (+ **TLS handshake** for HTTPS: certificate verification, key exchange)
3. HTTP request → server response
4. Browser parses HTML → renders (fetches CSS/JS/images, repeat)

HTTP Essentials

- **Methods**: GET (safe, idempotent) · POST (not idempotent) · PUT (idempotent replace) · PATCH (partial) · DELETE.
- **Status codes**: 200 OK · 201 Created · 301/302 redirect · 304 Not Modified · 400 Bad Request · 401 Unauthenticated · 403 Forbidden · 404 · 500 server error · 503 unavailable.
- **HTTP vs HTTPS**: 80 vs 443; HTTPS = HTTP over TLS (encryption + integrity + server authentication via certificates).
- **HTTP is stateless** → sessions via cookies/tokens.
- HTTP/1.1 keep-alive → HTTP/2 multiplexing over one TCP → HTTP/3 over QUIC (UDP).

Common Ports (memorize)

Port	Service	Port	Service
20/21	FTP	80	HTTP
22	SSH	110	POP3
23	Telnet	143	IMAP
25	SMTP	443	HTTPS
53	DNS	1521	**Oracle listener**
67/68	DHCP	3306	MySQL

Devices & Concepts

- **Hub** (L1, broadcasts everything) < **Switch** (L2, MAC table, per-port forwarding) < **Router** (L3, routes between networks).
- **Routing**: Distance vector (RIP — hop count, count-to-infinity problem) vs Link state (OSPF — Dijkstra on full topology); **BGP** = path-vector between autonomous systems (the Internet's routing protocol).
- **Error detection**: parity, checksum, **CRC** (data link). Error correction: Hamming code.
- **Flow control** (don't overwhelm receiver — sliding window) vs **congestion control** (don't overwhelm network).
- **Circuit switching** (dedicated path, telephony) vs **packet switching** (store-and-forward, Internet).
- **Firewall** (filter by rules), **proxy** (client-side intermediary), **reverse proxy/load balancer** (server-side), **VPN** (encrypted tunnel), **CDN** (edge caching).
- **Bandwidth** (capacity) vs **latency** (delay) vs **throughput** (actual achieved rate). Delay = transmission + propagation + queuing + processing.

---

6. Artificial Intelligence & Data Science

The Landscape

`AI` ⊃ `Machine Learning` ⊃ `Deep Learning`; Data Science = extracting insight from data (stats + ML + domain knowledge + engineering).

Learning Paradigms

Type	Data	Goal	Examples
Supervised	Labeled (X, y)	Predict y	Regression, classification
Unsupervised	Unlabeled	Find structure	Clustering (k-means), PCA, association rules
Semi-supervised	Few labels + many unlabeled	Leverage both	Label propagation
Reinforcement	Reward signal	Learn a policy	Q-learning, game agents

- **Classification** (discrete class: spam/not) vs **Regression** (continuous value: price).

Core Algorithms

Algorithm	Type	Key idea	Watch out
Linear Regression	Regression	Fit $y = wx + b$ minimizing MSE	Assumes linearity; sensitive to outliers, multicollinearity
Logistic Regression	Classification	Sigmoid $\rightarrow$ probability; log-loss	Linear decision boundary; still called "regression" (trap!)
k-NN	Both	Vote of k nearest neighbors	Lazy learner (no training); slow at predict time; scale features first!
Decision Tree	Both	Split on info gain / Gini impurity	Overfits deep trees $\rightarrow$ prune
Random Forest	Both	**Bagging** many trees + feature randomness	Less interpretable
Gradient Boosting (XGBoost)	Both	**Boosting**: sequential trees fix predecessors' errors	Tuning-sensitive; can overfit
SVM	Classification	Max-margin hyperplane; kernel trick for non-linear	Scaling matters; slow on huge data
Naive Bayes	Classification	Bayes' theorem + feature independence assumption	Assumption rarely true, still works (text)
k-Means	Clustering	Assign to nearest centroid, recompute, repeat	Must choose k (elbow method); sensitive to init (k-means++) and outliers
PCA	Dim. reduction	Project onto top-variance eigenvectors	Standardize first; components lose interpretability

- **Bagging vs Boosting**: parallel independent models on bootstrap samples reduce **variance** (Random Forest) vs sequential error-correcting models reduce **bias** (AdaBoost/XGBoost — more overfit-prone).

Bias–Variance Tradeoff

- **High bias** = underfitting (too simple; bad on train AND test) $\rightarrow$ add features/complexity.
- **High variance** = overfitting (memorized train; bad on test) $\rightarrow$ more data, regularization, simpler model, cross-validation.
- **Regularization**: **L1/Lasso** (drives weights to zero $\rightarrow$ feature selection) vs **L2/Ridge** (shrinks weights smoothly). Dropout/early stopping in neural nets.
- Total error = bias^2 + variance + irreducible noise.

Evaluation Metrics

Confusion matrix: TP, FP (Type I error — false alarm), FN (Type II — miss), TN.

Metric	Formula	Use when
Accuracy	(TP+TN)/all	Balanced classes only — misleading on imbalanced data (99% "no-fraud" model)
Precision	TP/(TP+FP)	FP costly (spam filter flagging real mail)

Metric	Formula	Use when
Recall (Sensitivity)	TP/(TP+FN)	FN costly (cancer screening, fraud)
F1	$2 \cdot P \cdot R/(P+R)$	Balance P & R; imbalanced data
ROC-AUC	Area under TPR-vs-FPR curve	Threshold-independent ranking quality
RMSE / MAE	Regression error	RMSE punishes big errors more
R^2	1 − SS_res/SS_tot	Variance explained

- **Train/validation/test split** (e.g., 70/15/15); **k-fold cross-validation** for robust estimates. **Never** tune on the test set.
- **Data leakage**: information from test/future leaks into training (e.g., scaling before splitting) → inflated scores. Fit preprocessing on train only.

Neural Networks & Deep Learning

- **Neuron**: weighted sum + bias → activation. Activations: **ReLU** (default hidden; avoids vanishing gradient, cheap), sigmoid (binary output), tanh, **softmax** (multiclass output).
- **Training**: forward pass → loss (cross-entropy / MSE) → **backpropagation** (chain rule gradients) → **gradient descent** update (`w := w − α·∂L/∂w`). Optimizers: SGD, momentum, **Adam**.
- **Learning rate**: too high diverges, too low crawls.
- **Vanishing/exploding gradients**: deep nets + sigmoid/tanh; fixes: ReLU, batch norm, residual connections, gradient clipping.
- **Architectures**: **CNN** (convolutions + pooling — images; parameter sharing, translation invariance), **RNN/LSTM/GRU** (sequences; LSTM gates fight vanishing gradients), **Transformer** (self-attention, parallelizable — basis of BERT/GPT/LLMs; "Attention Is All You Need").
- **Epoch** (full pass over data) vs **batch** (subset per update) vs **iteration** (one update).
- **Transfer learning**: start from pretrained model, fine-tune on your task — standard when data is limited.

Classical AI (exam favorites)

Search	Complete?	Optimal?	Notes
BFS	Yes	Yes (unit costs)	O(b^d) space — memory hog
DFS	No (infinite paths)	No	O(bm) space
Uniform Cost	Yes	Yes	Dijkstra flavor
A*	Yes	Yes if h **admissible** (never overestimates)	f(n) = g(n) + h(n)
Hill climbing	No	No	Stuck at local maxima/plateaus; fix: random restarts, simulated annealing
Minimax (+α–β pruning)	Game trees	Optimal vs rational opponent	α–β prunes without changing result; best-case O(b^(d/2))

- **Turing test**: machine indistinguishable from human in conversation.
- **Agent types**: simple reflex → model-based → goal-based → utility-based → learning agent.

Data Science Workflow & Statistics

- **Pipeline**: problem framing → collection → **cleaning** (missing values: drop/mean/median/model imputation; outliers: IQR rule — outside $Q1-1.5 \cdot IQR$ / $Q3+1.5 \cdot IQR$) → EDA → feature engineering → modeling → evaluation → deployment/monitoring.
- **Feature scaling**: standardization ($z = (x-\mu)/\sigma$) vs min-max normalization — required for k-NN, SVM, k-means, gradient descent; NOT needed for trees.
- **Encoding**: one-hot (nominal) vs label/ordinal encoding (ordered). Pitfall: label-encoding nominal categories injects fake order.
- **Central Limit Theorem**: sample means → normal distribution as n grows, regardless of population shape.
- **Hypothesis testing**: p-value = P(data this extreme | H_0 true); reject H_0 if $p < \alpha$ (0.05). p-value is NOT the probability H_0 is true (classic trap).

- **Correlation ≠ causation**; Pearson $r \in [-1, 1]$ measures *linear* association only.
- **Mean vs median**: median robust to outliers/skew (use for income, response times).
- **Imbalanced data fixes**: resampling (SMOTE/undersampling), class weights, threshold tuning, use F1/AUC not accuracy.
- **Curse of dimensionality**: distances become meaningless in high dimensions → k-NN/k-means degrade; use PCA/feature selection.

Cross-Subject Rapid-Fire (last 10 minutes)

Confusable pair	One-line difference
Overloading / Overriding	Same class, different params, compile-time / subclass, same signature, runtime
Abstract class / Interface	Shared code + state, single / pure contract, multiple
Process / Thread	Own memory / shared memory
Mutex / Semaphore	Ownership lock / signaling counter
Deadlock / Starvation	Circular mutual block / indefinite unfairness
Paging / Segmentation	Fixed physical / variable logical; internal vs external fragmentation
DELETE / TRUNCATE	DML, rollback, WHERE / DDL, fast, all rows
WHERE / HAVING	Before grouping / after aggregation
Clustered / Non-clustered index	Data stored in order / separate pointer structure
TCP / UDP	Reliable, ordered, slower / fast, best-effort
Switch / Router	L2, MACs, one network / L3, IPs, between networks
Flow / Congestion control	Protect receiver / protect network
Precision / Recall	Of predicted positives, how many right / of actual positives, how many found
Bagging / Boosting	Parallel, cuts variance / sequential, cuts bias
Classification / Regression	Discrete label / continuous value
L1 / L2 regularization	Sparse (feature selection) / smooth shrinkage
BFS / DFS	Queue, level-wise, shortest unweighted path / stack, deep-first, less memory
Stack / Queue	LIFO / FIFO
Compile-time / Runtime error	Syntax & types / logic & environment (null deref, divide by zero)